

AI Worker Integration Roadmap for Divyansh

Welcome to the AI-Backend Integration Guide! This document is prepared to help you connect your AWS EC2 Python AI Worker seamlessly with the AURIX backend.

Because we are using a Decoupled Microservice Architecture, you do not need to connect directly to the PostgreSQL database. Instead, you will communicate with my system via the Upstash Redis Queue and REST API Webhooks.

Here is your exact execution loop roadmap:

Phase 1: The Polling Loop (Redis)

When a user triggers a scan (either via a GitHub URL or a Zip upload), my API catches it and drops a "Job Ticket" into our shared Upstash Redis instance.

Your Task: Write a Python background daemon that checks this queue constantly.

- **Connection:** Connect to the `UPSTASH_REDIS_URL` using `redis-py`. (Ask me for the connection string in our group chat).
- **Queue Name:** `aurix_scan_queue`
- **Command:** Use `BRPOP aurix_scan_queue 5` (blocks for 5 seconds to reduce network chatter).

What the Job Ticket looks like (JSON):{

"scan_id": "a1b2c3d4-e5f6-7g8h",

"url": "https://github.com/user/repo",

"user_id": "9z8y7x6w-..."

}

(If it's a VS Code upload, it will contain `storage_path` instead of `url`).

When you pop this item, your LangGraph State Machine orchestrator should officially begin the AI triage and exploit generation process!

Phase 2: RAG Threat Intel Context (Optional but Recommended)

Before you feed the AST slice into your Logic Agent, you should ground its reasoning in factual OWASP data.

Instead of building your own Vector DB, I have already built it into my API using Supabase `pgvector`!

- **Endpoint:** `POST /api/internal/threat-intel/search`
- (Note: I will provide the live Render URL once I deploy)
- **Body:**

```
{  
  
  "query_embedding": [0.012, -0.045, 0.088, ...] // (Your 384-dimensional  
vector)  
  
}
```

- **Response:** A list of highly relevant markdown strings from OWASP guidelines that you can inject directly into your Logic Agent's system prompt.

Phase 3: The Webhook Return (Final Delivery)

Once your LangGraph workflow reaches **Node 7 (State Reducer)**, your Docker containers have verified the bugs, and you have compiled the final list of `verified_reports`, you must send this massive payload back to my backend so I can save it to PostgreSQL.

- **Endpoint:** `POST /api/internal/webhook/scan-complete`
- **Headers:**

```
{  
  
  "Authorization": "Bearer aurix-dev-token",  
  
  "Content-Type": "application/json"  
}
```

(Notice the hardcoded `aurix-dev-token`—this is our internal server-to-server security key).

Critical JSON Contract

You **must** use Pydantic `with_structured_output()` to guarantee your LLM outputs this exact structure, otherwise my database will reject the insertion.

Request Body Schema:

```
"scan_id": "a1b2c3d4-e5f6-7g8h", // Must strictly match the ID from Redis!  
  
"summary": {  
  
  "total_findings": 2,  
  
  "neutralized_count": 0  
  
},  
  
"findings": [  
  
  {  
  
    "rule_id": "OWASP-A01",  
  
    "tool": "Aurix-LangGraph",
```

```
"category": "Authentication",

"title": "Hardcoded JWT Secret",

"description": "Found a hardcoded secret in auth.js",

"severity": "CRITICAL",

"cvss": 9.8,

"file": "src/auth.js",

"line": 14,

"evidence": "const jwt_secret = 'super-secret-key'",

"fix": "Use environment variables.",

"wargame_status": "EXPLOITED",

"ai_reasoning": "The secret is plaintext and easily extractable via AST...",

"poc_script": "import jwt\nprint(jwt.encode({'user': 'admin'},\n'super-secret-key'))",

"patch_code": "const jwt_secret = process.env.JWT_SECRET;",

"verified": true

}

]

}
```